UNIVERSITY OF AUCKLAND
Waipapa Taumata Rau
NEW ZEALAND

SCIENCE

COMPSCI 714
Lectutorial 3: Design choices & Hyperparameter tuning

Normalisation, Learning rate, Faster Optimisers & Hyperparameter tuning

Instructor: Thomas Lacombe

Learning outcomes

- **Explain** the role of input normalisation and its impact on training stability and convergence.
- **Explain** the principles of Batch Normalisation and how it affects training dynamics.
- **Explain and compare** common approaches to learning rate tuning.
- **Explain and compare** optimisation algorithms, including Momentum, AdaGrad, RMSProp, and Adam.
- **Implement and experiment with** neural network training pipelines in PyTorch by configuring activation functions, weight initialisation, regularisation methods, normalisation, learning rates, and optimisation algorithms, and **evaluate** their impact on training stability, convergence speed, and generalisation performance.

Disclaimer/Acknowledgment:

Some of the following slides are reusing and adapting some of the content created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley.

All CS188 materials are available at http://ai.berkeley.edu.

Some content is also reused from the CS230 Stanford course slides created by Andrew Ng, available at https://cs230.stanford.edu/syllabus/.

The code in the notebook is adapted from the GitHub repository: ageron/handson-mlp.

Extra resources

If you want to go further:

- *Hands-on Machine Learning with Scikit-Learn and PyTorch*, Géron A.
  - Chapters 9, 10 and 11.

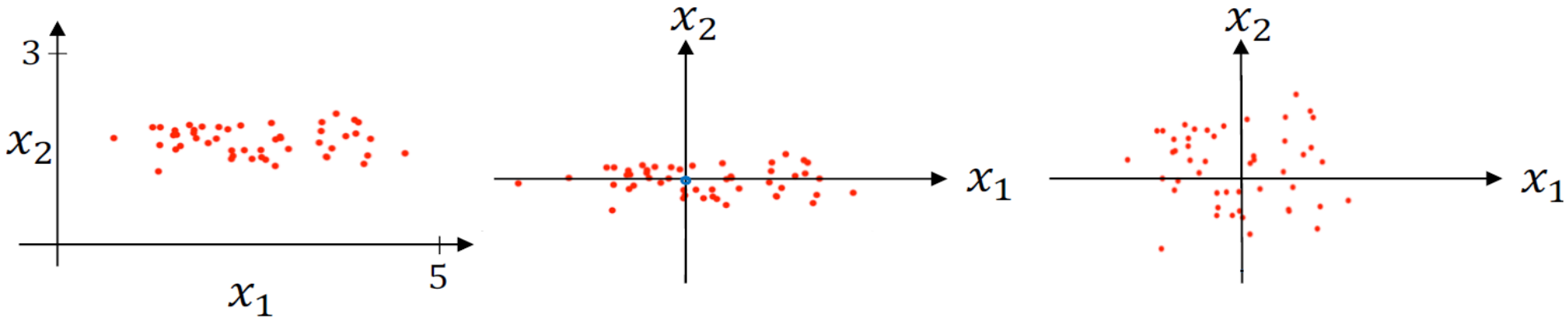
- *Deep Learning*, Goodfellow I. et al.
  - Part II Deep Networks, chapters 7, 8 and 11.

Activation functions, weights initialisation & regularisation

Normalising inputs

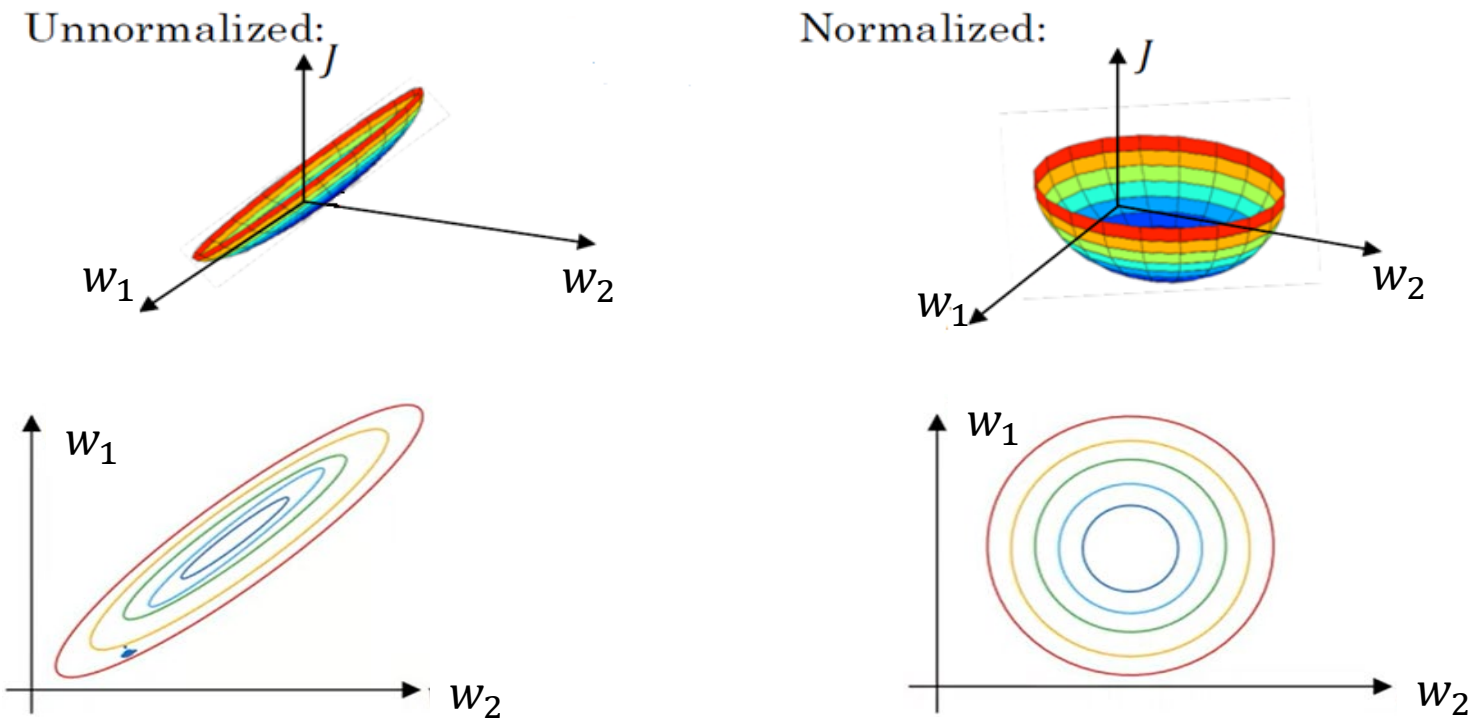
- **Normalising inputs of a NN (standardisation)**:

$$x \leftarrow \frac{x - \mu}{\sigma}$$



Normalising inputs

- **Why normalising inputs?**



→Makes training faster (e.g., less “zig-zag”)

→Reduces risk of vanishing/exploding gradients.

Images source: Andrew Ng

Batch normalisation

- Batch norm = normalising the input of each layer to enable faster training in each layer.

1. Computes the mean of the layer's inputs: μ
2. Computes the variance of the layer's inputs: σ^2
3. Compute the normed input value of each neuron in the layer:

$$z_{norm} = \frac{z - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$

4. Compute: $\tilde{z} = \gamma * z_{norm} + \beta$ (γ and β are learnable parameters)

- Running mean and variance are calculated during training and used at prediction time.

Batch normalisation - Regularisation effect

- Batch norm uses the mean and variance of mini-batches, and not the entire dataset.
- It introduces some noise in the neuron values, therefore in the activation values.

→This leads to some regularisation effect.

- Similar effect than using dropout.

- Also reduces the influence of weight initialisation!

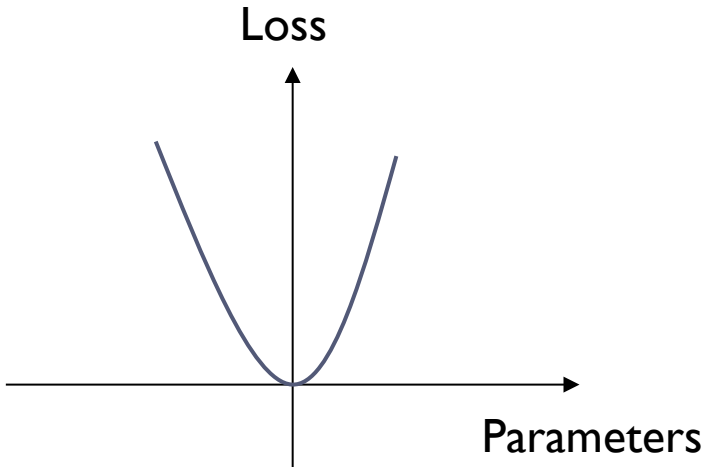
How to improve learning? Learning rate

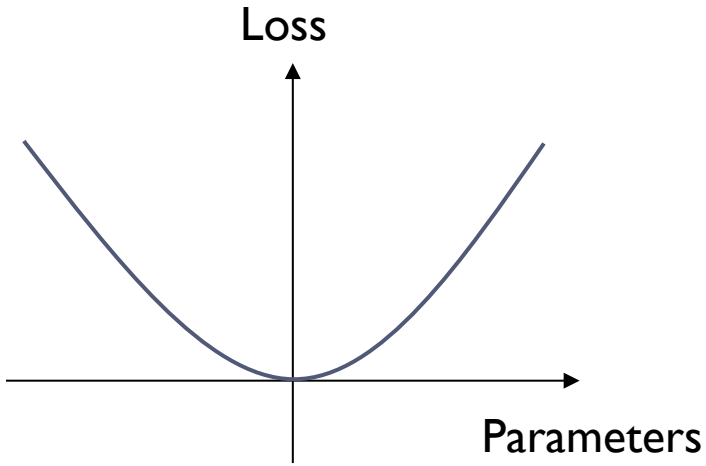
$$w \leftarrow w - \alpha * \nabla_w g(w)$$

- "One hyperparameter to rule them all."
  - Often seen as the most important hyperparameter to tune.
- Hard to know in advance what will be the best value.
  - Trial/error (common range: $10^{-6} < \alpha < 1.0$)
  - Grid/random/etc. search, optimisation techniques.

Choosing a learning rate

- Learning rate value vs curvature of the loss function





- The right learning rate depends on the slope!

- If α too small: learning will be slow and risk of local minima.
- If α too big: risk of overshooting (especially if slope is steep).

Learning rate decay schedule

- A common practice: decaying learning rate as learning progresses.

- Why?
  - → Cost function less steep as you come close to an optimum.
  - → Decaying the learning rate allows to take smaller steps when approaching the optimum.

- Finding the right decay schedule is non-trivial.
  - Time-based: Linear decay, exponential decay?
  - Step-based: how much to drop every how many epochs?

https://towardsdatascience.com/learning-rate-schedules-and-adaptive-learning-rate-methods-for-deep-learning-2c8f433990d1
https://machinelearningmastery.com/learning-rate-for-deep-learning-neural-networks/

Learning rate schedules

- **Adaptive learning rates schedules**
  - Monitors the model's performance and adapt the learning rate in response.
  - Common practice: reduce learning rate when performance plateaus.

- **Additional learning rate schedules**
  - **Learning-rate warm-up (early training)**
    - Gradually increase learning rate from a small value to a target value during the first few epochs of training
    - Large gradients + large learning rate early in training can lead to large unstable updates
  - **Cyclical learning rates and warm restarts**
    - Periodically increase and then decrease the learning rate during training
    - Cyclical schedules vary the learning rate within a fixed range
    - Warm restarts reset it to a higher value at scheduled times
    - Can help the optimiser escape poor local minima or sharp valleys

Refer to *Hands-On Machine Learning with Scikit-Learn and PyTorch*, Chapter 11, section "Learning Rate Scheduling" (pp. 398-405) for more examples of learning rate schedules and practical advice.

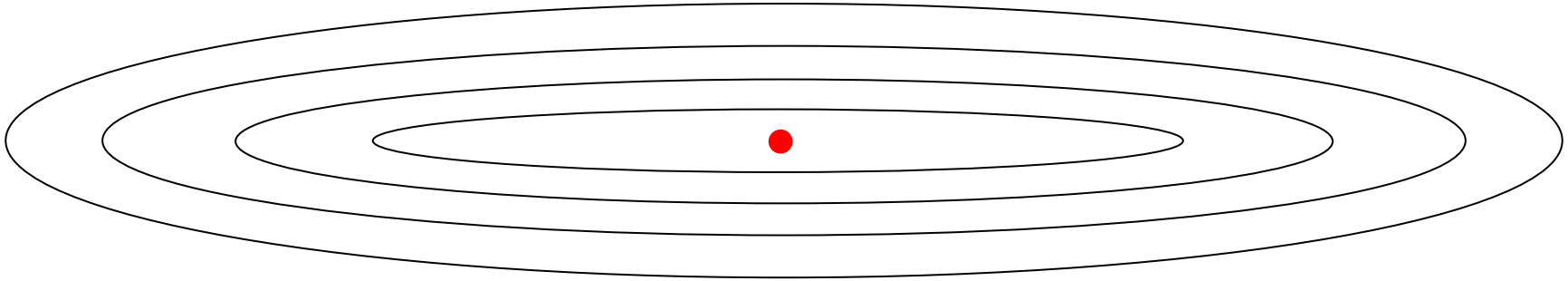
Hyperparameter tuning, batch-norm and learning rate schedule

Optimisers with adaptive learning rate

- Standard GD uses a fixed learning rate value or a schedule.
- Learning rate schedules rely on tuning and careful choices
- Other gradient optimisation algorithms implements adaptive learning rates.
- Adagrad, RMSProp, AdaDelta, ADAM, etc.
- Adaptive learning rate strategies generally outperform fixed and not well tuned learning rates.

Oscillations in learning

- **Problem**: standard GD oscillations make training slower and increase the risk of overshooting if the learning rate is too high.

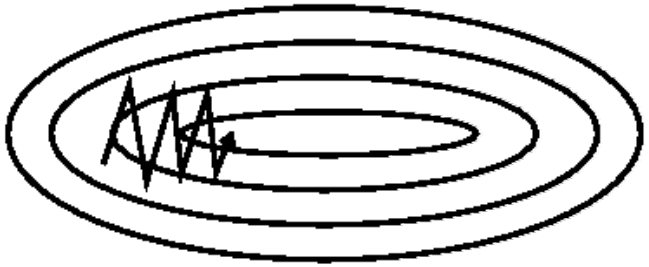


Add momentum

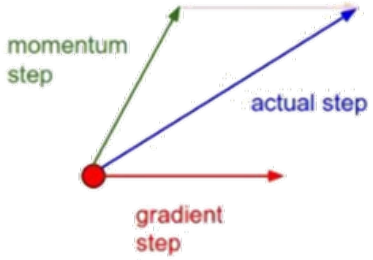
- **Momentum mechanism**: includes an exponentially decaying weighted average of previous gradients in the weight update.
  - Usually used with mini-batch Gradient Descent.

$$w \leftarrow w - \alpha \nabla_w g(w)$$

Without momentum



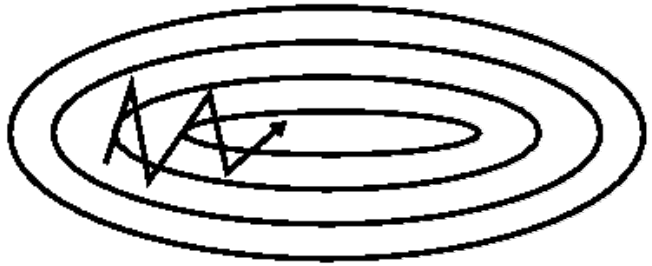
Momentum update



$$v_t = \beta v_{t-1} + (1-\beta)\nabla_w g(w)$$

$$w \leftarrow w - \alpha v_t$$

With momentum



Images source : https://cs231n.github.io/neural-networks-3/, https://www.willamette.edu/~gorr/classes/cs449/momrate.html

AdaGrad

- **Ada**ptive (sub)**Grad**ients algorithm: decays the learning rate based on the sum of previous squared gradients for each individual weight.

$$v_t = v_{t-1} + (\nabla_w g(w))^2$$

$$w \leftarrow w - \frac{\alpha}{\sqrt{v_t + \varepsilon}} \nabla_w g(w)$$

- Adapts the learning rate for each variable/dimension (i.e., weight).
- Reduces the influence of weights with high gradients and increase the influence of weights with lesser gradients (infrequent updates).

Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization.

AdaGrad

- **Problem with Adagrad**: it is very slow and could stop before reaching the minimum!

- Caused by sum of squared gradients monotonically increasing.

$$v_t = v_{t-1} + (\nabla_w g(w))^2$$

$$w \leftarrow w - \frac{\alpha}{\sqrt{v_t + \varepsilon}} \nabla_w g(w)$$

RMSProp

- **R**oot **M**ean **S**quared **Prop**agation algorithm: extension of Adagrad, introducing a decaying sum of squared gradients.

$$v_t = \beta v_{t-1} + (1-\beta)(\nabla_w g(w))^2$$

$$w \leftarrow w - \frac{\alpha}{\sqrt{v_t + \varepsilon}} \nabla_w g(w)$$

- Keeps the sum of squared gradients under control by applying a decay factor.
- RMSProp does not suffer from problem mentioned for Adagrad.

Lecture 6e rmsprop: Divide the gradient by a running average of its recent magnitude, Geoffrey Hinton

Adam

- **Adaptive Moment Estimation** algorithm: combines momentum and RMSprop to go even faster!

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla_w g(w) \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla_w g(w))^2$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

$$w \leftarrow w - \frac{\alpha}{\sqrt{\hat{v}_t + \epsilon}}\hat{m}_t$$

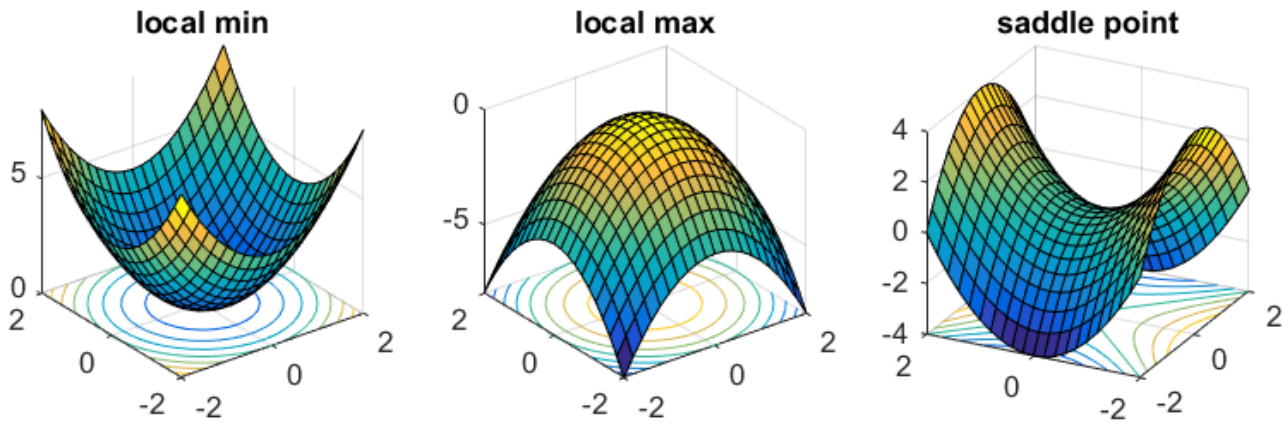
- Adam usually achieves good result fast on a good range of tasks → safe as a first choice.
- Pratice tip: AdamW recommended instead of Adam + L2 regularisation (decouples weight decay from gradient updates)

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization.
Loshchilov, I., & Hutter, F. (2017). Decoupled weight decay regularization

More optimisers: *Hands-On Machine Learning with Scikit-Learn and PyTorch*, Chapter 11, section "Faster Optimizers" (pp. 389-398)

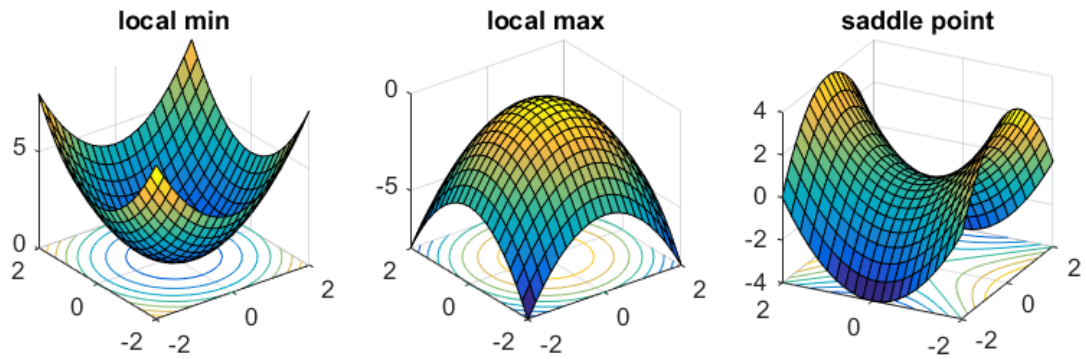
When are ADAM & co. useful?

- **Saddle points / Plateaus**



Local optima?

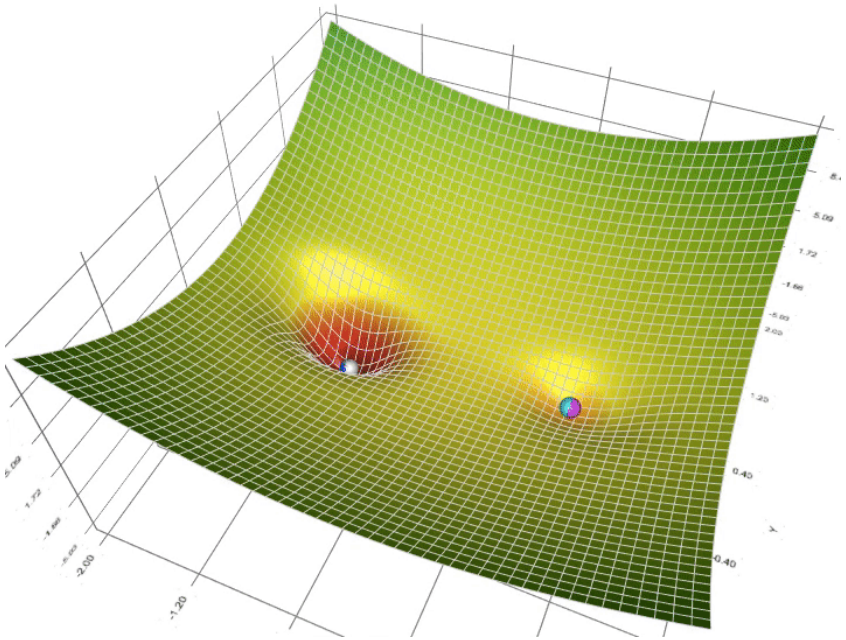
- Not likely to encounter a local optima with a large NN.
- High number of parameters → high number of dimensions for the cost function.
- A local optima (e.g., minima) require convex/concave shape in every dimension, which is very unlikely.
- More likely to get saddle points.



Visualisation tool

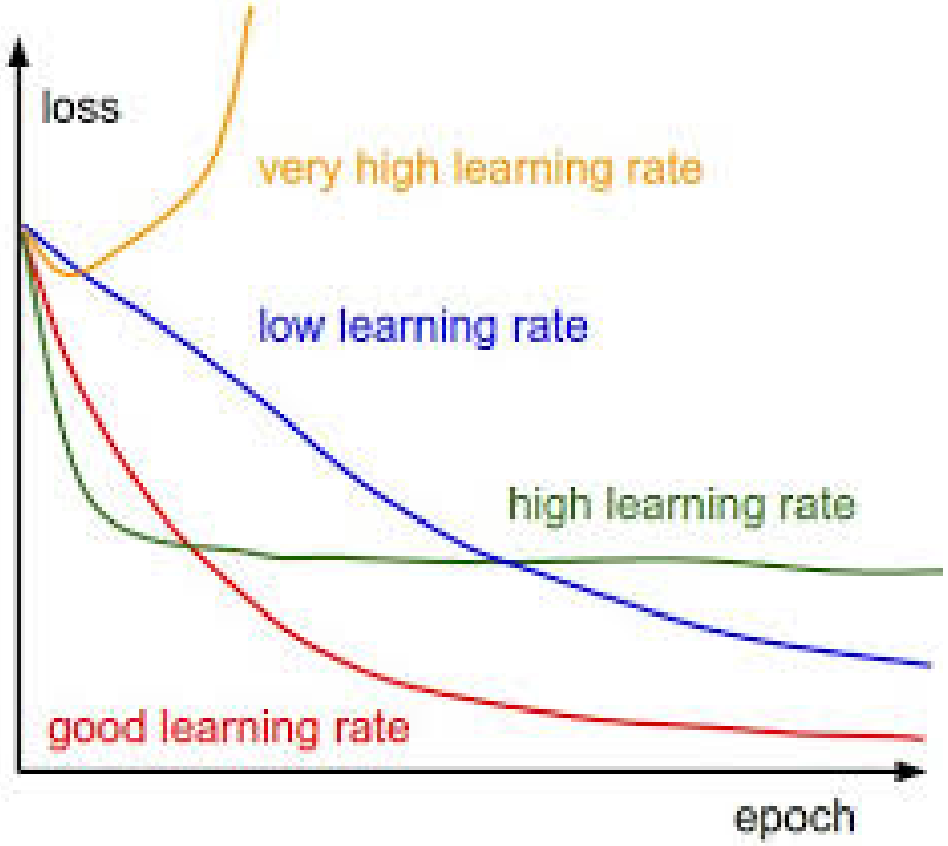
- A visual explanation of optimisation algorithms with a visualisation tool:

https://towardsdatascience.com/a-visual-explanation-of-gradient-descent-methods-momentum-adagrad-rmsprop-adam-f898b102325c



Faster optimisers

Conclusion


loss
very high learning rate
low learning rate
high learning rate
good learning rate
epoch